

Guía de estudio. Algoritmos de ordenación

Esta guía sirve para preparar el cuestionario. No es el solucionario: explica cómo se mueve cada algoritmo y qué tienes que mirar para decidir. La respuesta la sacas tú al final, con las preguntas de comprobación.

Lista de ejemplo que se usa en varios sitios: [4, 1, 3, 2].

1. Lo que hay que tener claro antes de estudiar algoritmo por algoritmo

Complejidad

La complejidad cuenta cómo crece el trabajo cuando crece el número de elementos, n . No es el tiempo en segundos de un ordenador concreto.

Notación	Idea	Si n pasa de 1.000 a 1.000.000
$O(1)$	Hace un trabajo fijo, da igual el tamaño	Sigue igual
$O(\log n)$	Parte el problema por la mitad una y otra vez	Crece muy poco
$O(n)$	Una pasada por la lista	Unas mil veces más
$O(n \log n)$	Varias pasadas, del orden de « n por el número de mitades»	Unas veinte mil veces más, no un millón
$O(n^2)$	Compara, en la práctica, casi cada elemento con casi todos los demás	Un millón de veces más: del orden de un billón de operaciones

Tres casos:

- **Mejor caso:** la entrada más favorable. A veces es «la lista ya está ordenada».
- **Peor caso:** la entrada que más trabajo obliga a hacer. A menudo es la lista al revés, o un pivote siempre malo.
- **Caso promedio:** lo habitual con datos mezclados, sin una trampa concreta.

Un algoritmo puede tener mejor caso $O(n)$ y peor caso $O(n^2)$. Otro puede costar lo mismo en los tres casos. Que el tiempo cambie o no cambie es la complejidad. Es otra cosa distinta de la estabilidad.

Estabilidad

Un algoritmo es **estable** cuando dos elementos con la misma clave conservan el orden en el que llegaron.

Imagina dos fichas con el mismo número, 2a y 2b, y 2a estaba delante. Después de ordenar, un algoritmo estable deja 2a delante de 2b. Uno que no es estable puede darles la vuelta.

Eso importa al ordenar por varios criterios. Si ya ordenaste por cliente y ahora ordenas por fecha, un algoritmo estable no deshace el orden de los pedidos que caen en la misma fecha.

Para decidirlo, mira **cómo mueve los datos**:

- Si solo cambia elementos vecinos que están desordenados, o inserta un elemento sin saltar por encima de otro igual, el orden de los iguales se conserva.
- Si intercambia elementos que no están al lado (un salto hasta el mínimo, un pivote, un padre con un hijo lejano del heap), dos iguales pueden cambiar de sitio.

Memoria y volumen de datos

Hay dos preguntas distintas y las dos salen en el cuestionario:

1. **¿Cuánto trabajo hace?** Un coste $O(n^2)$ se vuelve inviable con millones de datos, aunque quepan en memoria. Un coste $O(n \log n)$ aguanta mucho mejor el crecimiento.
2. **¿Caben los datos en la RAM?** Varios algoritmos de coste $O(n \log n)$ necesitan la lista entera en memoria. Si no cabe, el que está pensado para ese caso trabaja por bloques: ordena trozos que sí caben, los guarda en disco y los fusiona leyendo solo un bloque de cada trozo.

«Big Data», en este tema, apunta a las dos cosas: mucho volumen y, cuando se dice que no cabe en RAM, almacenamiento en disco.

Divide y vencerás

Divide el problema en partes más pequeñas, las resuelve y combina el resultado. Partir por la mitad, o separar menores y mayores que un pivote, es divide y vencerás. Comparar vecinos en un bucle no lo es.

2. Bubble Sort

Recorre la lista comparando **elementos adyacentes**: el 1.º con el 2.º, el 2.º con el 3.º, y así. Si están en el orden incorrecto, los intercambia. Los valores grandes van avanzando hacia el final, como burbujas. No empareja el primero con el último.

La versión clásica usa **dos bucles anidados**: uno cuenta las pasadas y el otro recorre los vecinos que aún pueden estar desordenados.

Con [4, 1, 3, 2], la primera pasada hace $4 \leftrightarrow 1$, luego $4 \leftrightarrow 3$, luego $4 \leftrightarrow 2$. El 4 ya quedó al final. La siguiente pasada trabaja sobre [1, 3, 2].

En el peor caso (la lista al revés) hay que ir colocando casi todos los pares. El número de comparaciones e intercambios crece como n^2 . Por eso, al duplicar los datos, el trabajo se multiplica aproximadamente por cuatro, y con millones de registros deja de ser viable.

Si se añade una parada —«si una pasada completa no ha intercambiado nada, la lista ya está ordenada»—, el mejor caso es una sola pasada: proporcional a n . Sin esa parada, seguiría haciendo todas las pasadas aunque la lista ya estuviera bien.

Para ordenar de mayor a menor no cambia el esquema de los bucles. Cambia la comparación: se intercambia cuando el de la izquierda es el menor, en lugar de cuando es el mayor.

Casi no se usa en sistemas reales. Se enseña porque se ve muy bien: sirve para introducir comparaciones, intercambios y complejidad antes de pasar a métodos mejores. Tendría sentido en un ejemplo de clase o en una lista tan pequeña que el tiempo no importa.

3. Selection Sort

En cada pasada busca el **mínimo de todo el tramo que aún no está ordenado** y lo coloca al principio de ese tramo, con un intercambio. Luego repite con el resto.

Con [4, 1, 3, 2] la primera pasada encuentra el 1 (está en la segunda posición) y lo cambia con el 4. Queda [1, 4, 3, 2]. La idea es simple de programar: un bucle que busca el mínimo y un intercambio.

No mira solo vecinos. Aunque la lista ya esté ordenada, cada pasada recorre entero el tramo restante para estar seguro de cuál es el mínimo. No hay una comprobación del estilo «si no he cambiado nada, paro». Por eso el orden de entrada no le ahorra comparaciones: el mejor caso, el promedio y el peor caso recorren la misma cantidad de trabajo, del orden de n^2 .

El número de **intercambios** sí es bajo: como máximo uno por posición, es decir $n-1$. Bubble Sort puede intercambiar en casi cada comparación. Esa es la ventaja de Selection Sort cuando escribir en memoria es muy caro.

No aprovecha una lista casi ordenada. Con millones de datos, el coste cuadrático lo deja fuera.

Tampoco es estable. El mínimo puede estar lejos y el intercambio salta por encima de otro elemento igual. Prueba con [2a, 2b, 1]: el 1 se cambia con 2a y los dos doses quedan en orden distinto del original.

Frente a Insertion Sort, los dos usan solo memoria extra constante y ordenan sobre la propia lista. La diferencia está en el movimiento: Selection elige el mínimo y lo coloca de un intercambio; Insertion toma el siguiente elemento y lo desplaza hacia atrás hasta su hueco. Insertion sí se acelera si los datos ya vienen ordenados.

En listas diminutas, un algoritmo simple puede ganar a uno teóricamente mejor porque tiene menos trabajo auxiliar. Aun así, para pocas posiciones suele encajar mejor Insertion Sort. En listas grandes encaja un algoritmo de coste $n \log n$.

4. Insertion Sort

Se parece a **ordenar una mano de cartas**: la parte izquierda ya está ordenada y la siguiente carta se inserta en su sitio, desplazando hacia la derecha las que son mayores.

Con [4, 1, 3, 2], el 1 retrocede delante del 4; luego el 3 se coloca entre 1 y 4; luego el 2 se coloca entre 1 y 3.

El coste real depende de **cuántas posiciones retrocede cada elemento**. Si la lista está al revés, cada elemento nuevo recorre casi todo lo que ya estaba ordenado: peor caso $O(n^2)$. Si ya está ordenada, cada elemento se compara con el anterior y no se mueve: una pasada, $O(n)$. Si está casi ordenada, cada uno se mueve poco y el tiempo se acerca a esa pasada lineal. Por eso un peor caso cuadrático no impide que, con datos favorables, sea muy rápido.

Es estable: un elemento solo adelanta a los que son estrictamente mayores. Dos iguales no se cruzan.

No es recomendable para grandes volúmenes, por el peor caso cuadrático. Sí se sigue usando con **listas pequeñas**, del orden de unas pocas decenas de elementos, porque hace poco trabajo extra. Timsort lo usa por dentro precisamente para esos tramos cortos.

5. Gnome Sort

También compara vecinos, así que se parece mucho a Bubble Sort. La diferencia es el movimiento, con **una sola posición** en lugar de dos bucles:

- si el elemento actual y el anterior están en orden, avanza un puesto;
- si no, los intercambia y **retrocede** un puesto.

Puede retroceder varias veces seguidas, mientras siga encontrando desorden. Esa marcha atrás es también lo que lo acerca a Insertion Sort: lleva un elemento hacia atrás hasta su sitio, pero con un solo índice y sin bucle anidado. El código queda corto y, en la práctica, hace más intercambios.

El nombre viene de un **gnomo de jardín** que ordena macetas de esa misma forma: un paso adelante si están bien, un cambio y un paso atrás si no. También se llamó **stupid sort**. No es bogosort: bogosort mezcla al azar hasta que, por casualidad, la lista queda ordenada.

En el peor caso (lista al revés) necesita del orden de n^2 intercambios, igual que Bubble Sort, aunque el camino por la lista sea distinto. Si la lista ya está ordenada solo avanza, y el coste es una pasada $O(n)$. No tarda siempre lo mismo.

Con un millón de registros el peor caso cuadrático es inaceptable. En sistemas reales casi no se elige: es un ejercicio para ver un bucle simple. Cada paso depende del anterior (avanzar o retroceder), así que no se reparte en trozos independientes para paralelizarlo sin cambiar el algoritmo.

6. Cocktail Shaker Sort

Es un Bubble Sort que **cambia de sentido**. Una pasada va de izquierda a derecha y coloca un valor grande al final. La siguiente va de derecha a izquierda y coloca un valor pequeño al

principio. El nombre sale de ese vaivén, el de una coctelera. También se llama ordenación de burbuja bidireccional.

No ordena los dos extremos en el mismo instante ni con dos bucles paralelos. Alterna. En un ciclo completo (ida y vuelta) queda fijado un elemento en cada extremo.

El problema que intenta resolver es el de las **tortugas**: un valor pequeño situado cerca del final, en Bubble Sort, solo avanza una posición por pasada hacia el principio. Al volver en sentido contrario, ese valor camina más deprisa. Los grandes siguen yendo hacia el final en la pasada de ida.

La operación básica sigue siendo comparar e intercambiar vecinos. Por eso el peor caso no cambia de orden: sigue siendo $O(n^2)$. La mejora reduce pasadas en algunos casos; no convierte el algoritmo en uno de tipo $n \log n$.

Si se detiene cuando una pasada no hace ningún intercambio, y la lista ya está ordenada, le basta una pasada: mejor caso $O(n)$. No sigue viajando de extremo a extremo sin necesidad. Sin esa parada, las pasadas completas seguirían aunque no hubiera nada que cambiar.

7. Quick Sort

Es divide y vencerás. Elige un **pivote**, separa el tramo en elementos menores que el pivote y elementos mayores, y repite lo mismo en cada parte. Los iguales pueden quedar a un lado o a otro según cómo esté escrito.

El pivote decide el rendimiento. Si deja dos partes de tamaño parecido, el número de niveles es pequeño (logaritmo de n) y en cada nivel se recorre la lista: el caso habitual es $O(n \log n)$. Si el pivote es siempre el mínimo o el máximo del tramo, una parte queda vacía y la otra se queda con casi todos los elementos. Eso ocurre, por ejemplo, con una lista ya ordenada si se elige siempre el primer elemento o siempre el último. Entonces el peor caso es $O(n^2)$.

Para hacer ese peor caso menos probable se evita elegir siempre un extremo. Sirven el elemento de la posición central, un elemento al azar, o la **mediana de tres** (por ejemplo el primero, el central y el último, y quedarse con el del medio de esos tres). Ninguna de estas ideas borra del todo el peor caso teórico; lo hacen mucho más raro.

No es estable. En la partición hay intercambios entre posiciones que no son vecinas, y dos elementos iguales pueden cambiar su orden relativo.

Algunas librerías no se quedan en Quick Sort si la recursión se hace demasiado profunda, que es la señal de que las particiones van mal. **Introsort** empieza con Quick Sort por su buena rapidez media y, si la profundidad pasa de un límite, cambia a Heap Sort, que garantiza $O(n \log n)$.

8. Merge Sort

También es divide y vencerás, de otra manera. Parte la lista **por la mitad** hasta llegar a listas de un solo elemento (esas ya están ordenadas) y después **fusiona** las mitades de dos en dos, siempre sacando el menor de los dos frentes.

Con 8 elementos hay 3 niveles de división, porque 8, 4, 2, 1. En general, partir n por la mitad una y otra vez da del orden de $\log n$ niveles, y en cada nivel se recorre toda la lista al fusionar. El coste es $O(n \log n)$.

La versión clásica hace siempre esas divisiones y esas fusiones. No mira al empezar si la lista ya estaba ordenada, ni se salta el trabajo cuando lo está. El mejor caso, el promedio y el peor caso coinciden. Una lista grande ya ordenada se divide y se vuelve a fusionar igual.

Es estable si, al fusionar, en caso de empate se toma primero el elemento que venía de la mitad izquierda. Así los iguales no se adelantan.

La fusión no puede escribir el resultado encima de datos que todavía no ha leído. Necesita un **array auxiliar de tamaño proporcional a n** . Esa memoria extra es su desventaja frente a Heap Sort y frente a la versión habitual de Quick Sort, que ordenan prácticamente sobre la propia lista. Y la versión clásica necesita los datos en memoria: no es el algoritmo de los bloques en disco.

Hadoop, en la fase de ordenación de MapReduce, usa una ordenación por mezcla de este estilo para grandes volúmenes. Merge Sort, como algoritmo, no necesita Hadoop para ejecutarse.

La versión clásica no aprovecha tramos que ya venían ordenados: los trocea hasta el elemento suelto. Quien sí los detecta y los fusiona tal cual es el *natural merge sort* y, más afinado, Timsort.

9. Timsort

Combina las dos ideas anteriores. Busca grupos consecutivos que ya están ordenados. Esos grupos se llaman **runs**. Si un run natural es demasiado corto, lo alarga con Insertion Sort hasta un tamaño mínimo, el **minrun**, que en la práctica queda entre 32 y 64 elementos según la

longitud de la lista. Luego junta los runs **fusionándolos como Merge Sort**. Si toda la lista es ya un único run, no hay nada que fusionar.

El nombre es el de su creador, **Tim Peters**. Es el algoritmo de ordenación por defecto de **Python** (sorted y list.sort). También está en otros lenguajes; en este tema la referencia es Python.

El peor caso sigue siendo el de la mezcla, $O(n \log n)$. El mejor caso es $O(n)$: una pasada basta para ver que todo el array ya es un run y devolverlo. Por eso puede ser lineal aunque Merge Sort clásico, que no busca runs, sea $n \log n$ también en el mejor caso.

Le favorece que los datos reales suelen venir **parcialmente ordenados** (fechas, ids, listas casi ordenadas). Cuanto más largos son los runs, menos fusiones hace. Tener los datos total o parcialmente ordenados reduce el trabajo. Llegar al revés, o no caber en memoria, no es lo que lo acelera.

Python lo usa en vez de Quick Sort por esas dos razones juntas: aprovecha el orden parcial y es **estable**. Quick Sort no garantiza la estabilidad, y la estabilidad importa si ordenas primero por un campo y después por otro.

Funciona en **un solo ordenador**, siempre que la lista quepa en RAM. Necesita la lista en memoria. Si el volumen no cabe, Timsort por sí solo no es la herramienta: hace falta una ordenación externa.

10. Heap Sort

Un **heap** es un árbol binario guardado, casi siempre, dentro del propio array. No hace falta crear objetos nodo con punteros: el padre y los hijos se calculan con el índice. Si el elemento está en la posición i , sus hijos están en $2i+1$ y $2i+2$.

En el **max-heap** que suele usar Heap Sort, cada padre es **mayor o igual que sus hijos**. El máximo está en la raíz. El algoritmo extrae esa raíz, coloca el último elemento en su lugar y rehace el heap. El siguiente máximo vuelve a quedar arriba. Repetir eso saca los elementos de mayor a menor.

Construir el heap cuesta una cantidad de trabajo proporcional a n , y no depende de si la lista venía ordenada. Después hay n extracciones, y cada una baja por la altura del árbol, que es $\log n$. El producto es $O(n \log n)$ en el mejor caso y en el peor. No existe una entrada «fácil» que se salte las extracciones, ni una entrada desfavorable que lo convierta en cuadrático.

Si todos los valores son iguales, el heap sigue siendo válido, el algoritmo termina bien y el coste no cambia de orden. No aparece un atajo. Otra cosa es la estabilidad: los intercambios del heap

no son entre vecinos, así que dos elementos con la misma clave (la misma prioridad, el mismo número) pueden salir en un orden distinto del de llegada. No garantiza el orden de llegada.

La misma estructura, usada como **cola de prioridad**, encaja cuando siempre hay que atender el más urgente: por ejemplo el orden de atención en un hospital. Si los datos llegan de uno en uno, insertar o extraer el más prioritario cuesta $O(\log n)$. No hace falta reordenar toda la lista en cada llegada.

La versión habitual construye el heap con **toda la lista en memoria**. Aunque el coste sea $O(n \log n)$, no es la elección cuando los datos no caben en RAM.

11. External Merge Sort

Es la variante de Merge Sort para cuando **la lista no cabe en RAM**.

1. Se corta el fichero en trozos que sí caben en memoria.
2. Cada trozo se ordena en RAM y se escribe ya ordenado en disco. Esos trozos ordenados son runs.
3. Se fusionan leyendo solo un bloque de cada run. En memoria nunca está el fichero entero: está el bloque con el que se trabaja. El grueso de los datos vive en disco.

El número de comparaciones sigue el de la mezcla, $O(n \log n)$. El tiempo real, con volúmenes enormes, depende también de las lecturas y escrituras en disco, que suelen ser lo más caro.

Sirve para cantidades masivas. Se puede paralelizar la ordenación de los trozos (cada uno es independiente) y también parte de la fusión.

El criterio para elegirlo es el tamaño frente a la RAM disponible. Si los datos caben, se ordenan en memoria (Timsort, Merge Sort, Heap Sort, etc.). Si no caben, se usa esta ordenación por bloques en disco. No depende de si los datos son texto o números, ni de si hay repetidos.

12. Cómo decidir en el cuestionario

Cuando leas una pregunta, clasifícala. Casi todas piden una de estas cosas.

Cómo se mueve

- ¿Compara vecinos, busca el mínimo, inserta como las cartas, retrocede un puesto, va y viene, parte con un pivote, fusiona mitades, usa un árbol o trabaja por bloques en disco?
- ¿Una pasada fija solo el final, o un ciclo de ida y vuelta fija los dos extremos?
- ¿El paso siguiente depende del anterior (avanzar o retroceder) o se puede repartir en trozos independientes?

Qué coste tiene

- Vecinos y, en el peor caso, casi todos los pares: $O(n^2)$.
- Siempre recorre el tramo restante, sin poder parar: el coste no baja aunque la lista esté ordenada.
- Puede parar cuando una pasada no cambia nada, o cada elemento ya está en su sitio: el mejor caso es $O(n)$.
- Parte por la mitad, o extrae n veces de un árbol de altura $\log n$: $O(n \log n)$, a menudo en todos los casos.
- El pivote siempre en un extremo: Quick Sort puede caer a $O(n^2)$. Un pivote central, al azar o mediana de tres lo hace menos probable.

Si aguanta mucho volumen

- $O(n^2)$ y un millón de registros no combinan.
- $O(n \log n)$ con la lista entera en RAM combina si los datos caben.
- Si no caben, el esquema es el de bloques en disco.

Si es estable

- Define primero qué significa: los iguales conservan su orden de llegada. No significa «tarda siempre lo mismo».
- Después mira si el algoritmo salta elementos o solo mueve vecinos / inserta sin cruzar iguales.

Datos concretos que son nombres o cifras, no un cálculo

- El otro nombre de Gnome Sort, y de qué imagen sale el nombre oficial.
- El nombre de pila del autor de Timsort, y en qué lenguaje es el algoritmo por defecto.
- Entre qué tamaños queda el minrun.
- Cómo se llama la técnica que cambia de Quick Sort a Heap Sort si la recursión se hunde.
- Qué sistema de procesamiento masivo usa una ordenación por mezcla en su fase de ordenación.

13. Comprueba si ya puedes responder

Contesta con tus palabras, sin mirar esta guía. Si te atascas, vuelve solo a la sección del algoritmo, no busques una solución escrita: aquí no está.

1. ¿Bubble Sort empareja extremos o vecinos? ¿Cuántos bucles anidados usa la versión clásica? ¿Es divide y vencerás?
2. ¿Qué pasa con su coste si la lista ya está ordenada y se detiene al no haber intercambios? ¿Y en el peor caso? ¿Sirve para millones de datos? ¿Para qué se enseña entonces? ¿Qué comparación hay que invertir para ordenar al revés?
3. ¿Qué coloca Selection Sort en cada pasada? ¿Cambia su número de comparaciones si la lista ya está ordenada? ¿Cuántos intercambios hace como máximo, y en qué le gana eso a Bubble Sort?
4. ¿Selection Sort es estable? Pruébalo con [2a, 2b, 1]. ¿En qué se diferencia de Insertion Sort al mover los datos y al aprovechar una lista casi ordenada?
5. ¿A qué gesto cotidiano se parece Insertion Sort? ¿Cuál es su peor caso y cuál es su mejor caso? ¿Es estable? ¿Por qué una lista casi ordenada lo acelera tanto? ¿Con qué tamaños se sigue usando?
6. ¿Qué hace Gnome Sort después de un intercambio? ¿Puede retroceder más de una posición? ¿A qué dos algoritmos se parece, y en qué? ¿Cuál es su otro nombre y de dónde sale «gnome»? ¿Su coste es siempre el mismo? ¿Tiene sentido con un millón de filas o en paralelo?
7. ¿En qué direcciones se mueve Cocktail Shaker Sort y de qué objeto sale el nombre? ¿Qué les pasa a los valores pequeños que están al final de la lista en Bubble Sort? ¿Ordena los dos extremos a la vez? Si ya está ordenada y hay parada por «cero intercambios», ¿cuántas pasadas necesita? ¿Por qué el peor caso no baja de orden?
8. ¿En qué estrategia se basa Quick Sort y en qué dos grupos deja el tramo? ¿Cuándo aparece su peor caso? ¿Qué formas de elegir el pivote lo hacen menos probable y por qué el pivote importa tanto? ¿Es estable? ¿Qué hace Introsort si la recursión se alarga demasiado?
9. ¿Cómo divide y cómo junta Merge Sort clásico? ¿Su mejor caso es distinto del peor? ¿Qué hace con una lista que ya estaba ordenada? ¿Necesita la lista en memoria? ¿Por qué gasta más RAM que Heap Sort o Quick Sort? ¿Es estable, y qué significa eso de verdad? ¿Qué relación tiene con Hadoop, en qué sentido? ¿Quién aprovecha los tramos que ya venían ordenados?
10. ¿Qué dos algoritmos están dentro de Timsort? ¿Cómo se llaman los grupos que busca y hasta dónde alarga uno corto? ¿Por qué se llama así y en qué lenguaje es el método por defecto? ¿Cómo junta esos grupos? ¿Por qué su mejor caso puede ser $O(n)$? ¿Por qué ese lenguaje lo prefiere a Quick Sort? ¿Le basta una sola máquina? ¿Le basta si los datos no caben en RAM? ¿Qué situación de los datos le quita trabajo?
11. ¿Heap Sort necesita toda la lista en memoria? ¿Cuál es su peor caso y por qué el mejor no es más barato? ¿Sirve cuando no hay RAM suficiente? ¿Para qué tipo de problema

encaja el heap, incluido un ejemplo de prioridades? Si todos los valores son iguales, ¿falla, se acelera o sigue igual? Si hay empates de prioridad, ¿conserva el orden de llegada? ¿Qué relación tiene un padre con sus hijos en un max-heap? ¿Hay que crear nodos con punteros? ¿Para qué sirve tener el extremo en la raíz? Si los datos llegan de uno en uno, ¿por qué no reordenas toda la lista cada vez?

12. ¿Cuándo se usa External Merge Sort en lugar del Merge Sort clásico? ¿Dónde está la mayor parte de los datos durante el proceso? ¿Qué coste tienen las comparaciones? ¿Se puede paralelizar algo? ¿Qué miras para elegir entre ordenar en memoria y ordenar en disco?
13. Entre Cocktail Shaker, Gnome, Selection y Merge, ¿cuál aguanta mejor que n crezca, y por qué los otros no? Si la lista ya está ordenada, ¿cuál de Bubble (con parada), Gnome y Selection sigue haciendo del orden de n^2 comparaciones? ¿Quién detecta zonas ya ordenadas? ¿Quién usa un árbol? ¿Quién recorre en los dos sentidos? ¿Quién deja el grueso de los datos en disco?